INTEGRATION LAYER V2.0

Runbook and API Reference

Step-by-step launch commands and every API

Version 2.0 | 28 September 2026

Contents: Runbook, Api Reference

Contents

Integration Layer V2.0: Runbook 3

1. Prerequisites . . . 3
2. Get the code and configure . . . 3
3. Build and launch . . . 3
4. Verify . . . 4
5. Open the applications . . . 4
6. Try it (5 minutes) . . . 4
7. Day-2 operations . . . 5
8. Use real Google BigQuery instead of the emulator . . . 5
9. Automated tests (optional) . . . 5
10. Troubleshooting . . . 6
11. Stop and reset . . . 6

Integration Layer V2.0: API Reference 7

1. Integration Layer . . . 7
2. Workflow payloads . . . 9
3. ConsumerApplication . . . 9
4. WWMG_Source . . . 9
5. MyCareHub . . . 9
6. Console backend (/api) . . . 10
7. Endpoint index (generated) . . . 10

Integration Layer V2.0: Runbook

Step-by-step commands to launch, verify, operate and stop every service. All commands run from the repository root unless stated otherwise.

1. Prerequisites

Requirement	Check	Notes
Docker Desktop 4.x (Engine 24+) with Compose v2	`docker compose version`	Docker must be running
At least 15 GB free disk	`df -h ~`	Docker makes its disk read-only when the Mac is nearly full (see §10)
Free ports	`lsof -i :8000,8001,8002,8003,8080,5432,9050`	stop V1 first: `cd ../ && docker compose down`
Git	`git --version`	
Python 3.12+ (optional)	`python3 --version`	only for the smoke test and the automated tests

2. Get the code and configure

```
git clone https://github.com/ArghaMukherjee/IntLayerV2.0.git
cd IntLayerV2.0

cp .env.example .env
# Replace every value. Quick way to generate secrets:
for k in POSTGRES_PASSWORD IL_API_DB_PASSWORD CONSUMER_DB_PASSWORD SCHEDULER_DB_PASSWORD \
         BI_READER_DB_PASSWORD CONSOLE_DB_PASSWORD CALLBACK_SIGNING_SECRET; do
  sed -i.bak "s/^$k=.*/$k=$(openssl rand -hex 16)/" .env
done
for k in WWMG_API_KEY MYCAREHUB_API_KEY ADMIN_API_KEY; do
  sed -i.bak "s/^$k=.*/$k=key-$(openssl rand -hex 16)/" .env
done
rm -f .env.bak
```

3. Build and launch

```
docker compose build                # builds the 5 service images
docker compose up -d                # starts postgres + bigquery, runs migrate once, then the services
docker compose ps                   # wait until everything is healthy / running
```

Start-up order is enforced: `postgres` (healthy) and `bigquery` → `migrate` (exits 0) → `il-api`, `il-scheduler`, `consumer-application` → `wwmg-source`, `mycarehub`, `console` (after il-api is healthy).

Expected `docker compose ps`:

Service	State
postgres	running (healthy)
bigquery	running
migrate	exited (0)
il-api, consumer-application, console	running (healthy)
il-scheduler, wwmg-source, mycarehub	running

4. Verify

```
# 4.1 Health
curl -s localhost:8000/health/ready     # {"status":"ready","database":"ok","bigquery":{"ok":true,...}}
curl -s localhost:8001/health           # workers: {"postgres":true,"bigquery":true}
curl -s localhost:8002/health; curl -s localhost:8003/health; curl -s localhost:8080/health

# 4.2 Migrations and BigQuery bootstrap
docker compose logs migrate | tail -12  # "migrations complete (4 new)" and "BigQuery ... ready"

# 4.3 End-to-end smoke test (both sources, both backends, callbacks, retries, backend switch)
python3 -m pip install httpx
python3 scripts/smoke_test.py           # reads keys from .env; prints "RESULT: ALL PASSED"
```

5. Open the applications

Application	URL	What to do
Console	http://localhost:8080	Dashboard, Analytics, Jobs, Payloads & send, Requests, Telemetry, Workflows. Theme toggle top-right.
MyCareHub	http://localhost:8003	Pick a member, request care, watch the status and notifications
Integration Layer API docs	http://localhost:8000/docs	Swagger UI (use the X-API-Key "Authorize" button)
ConsumerApplication API docs	http://localhost:8001/docs	
WWMG_Source API docs	http://localhost:8002/docs	
MyCareHub API docs	http://localhost:8003/docs	
PostgreSQL	`localhost:5432`, db `integration`	e.g. `docker compose exec postgres psql -U postgres -d integration`
BigQuery emulator	http://localhost:9050	e.g. `bq --api http://localhost:9050 query ...` or the Python client

6. Try it (5 minutes)

```
KEY=$(grep ^WWMG_API_KEY .env | cut -d= -f2)
# 6.1 Send an order as WWMG_Source (PostgreSQL workflow) and poll it
curl -s -X POST localhost:8000/v1/requests -H "X-API-Key: $KEY" -H 'Content-Type: application/json' -d '{
  "workflow_name":"order_validation","callback_url":"http://wwmg-source:8002/hooks/il",
  "payload":{"order_id":"ORD-1","customer_id":"C-1","currency":"EUR","items":[{"sku":"A","quantity":2,"unit_pric \
    e":10}]}}'
curl -s localhost:8000/v1/requests/<request_id> -H "X-API-Key: $KEY"

# 6.2 A MyCareHub member request (BigQuery workflow)
curl -s -X POST localhost:8003/api/care-requests -H 'Content-Type: application/json' \
  -d '{"member_id":"M-1001","request_type":"appointment","urgency":"routine"}'
curl -s 'localhost:8003/api/notifications?member_id=M-1001'
```

In the console:

- Jobs: start "Steady order traffic" (PostgreSQL) and "MyCareHub member traffic" (BigQuery) with Resume; use Run now on the manual jobs; pause/resume each ConsumerApplication queue and watch PENDING build up and drain.
- Payloads & send: pick a sample, Validate, choose Send as WWMG_Source or MyCareHub, Send now; the response panel shows the status code, the response body, the backend and live progress.
- Workflows: Edit schemas → change Storage backend → save; new requests go to the other store.
- Analytics: filter by backend; Refresh warehouse now loads both backends immediately.

7. Day-2 operations

```
docker compose logs -f il-api consumer-application il-scheduler      # follow logs
docker compose restart consumer-application                          # restart one service
docker compose up -d --build console                                 # rebuild + redeploy one service after a \
    code change
docker compose up -d --scale consumer-application=3                  # more consumers (remove its "8001:8001" \
    port mapping first)
docker compose exec postgres psql -U postgres -d integration -c \
  "SELECT status, count(*) FROM integration.request_metadata GROUP BY 1"      # PostgreSQL queue by status
```

Pause or resume consumption from the command line:

```
curl -s -X POST 'localhost:8001/worker/pause?backend=bigquery'      # or postgres; omit for both
curl -s -X POST localhost:8001/worker/resume
```

Switch a workflow's backend from the command line:

```
ADMIN=$(grep ^ADMIN_API_KEY .env | cut -d= -f2)
curl -s localhost:8000/v1/workflows/order_validation -H "X-API-Key: $KEY" > wf.json
python3 - <<'EOF'
import json; w=json.load(open("wf.json"))
body={k:w[k] for k in ("target_app","description","input_schema","output_schema","max_retries","is_active")}
body["storage_backend"]="bigquery"; json.dump(body, open("wf.json","w"))
EOF
curl -s -X PUT localhost:8000/v1/admin/workflows/order_validation -H "X-API-Key: $ADMIN" -H 'Content-Type: \
    application/json' -d @wf.json
```

8. Use real Google BigQuery instead of the emulator

1. In a GCP project: enable the BigQuery API.
2. Create a service account with BigQuery Data Editor and BigQuery Job User on the project (or Data Editor on the dataset only), and download a JSON key.
3. Save the key as `secrets/gcp-service-account.json` (the `secrets/` folder is git-ignored).
4. In `.env`: `BQ_PROJECT=<your-project-id>` and optionally `BQ_DATASET`, `BQ_LOCATION`.
5. Launch with the override: ``bash docker compose -f docker-compose.yml -f docker-compose.gcp.yml up -d --build docker compose logs migrate | tail -3 # creates the dataset + partitioned, clustered tables
6. Real BigQuery bills per query. The override polls every 20 s (`CONSUMER_BQ_POLL_SECONDS`), and idle pollers only run a cheap `SELECT … LIMIT 1` before any DML.

9. Automated tests (optional)

```
python3 -m venv .venv && . .venv/bin/activate && pip install -r requirements-dev.txt
docker compose up -d postgres bigquery
TEST_PG_ADMIN_URL=postgresql://postgres:$(grep ^POSTGRES_PASSWORD .env | cut -d= -f2)@localhost:5432/postgres \
TEST_BQ_EMULATOR=http://localhost:9050 pytest            # 46 tests; without TEST_BQ_EMULATOR the BigQuery \
    tests are skipped
```

The tests create and drop their own database, login users and BigQuery dataset.

10. Troubleshooting

Symptom	Cause	Fix
`read-only file system` from Docker; containers turn unhealthy	Mac disk almost full, Docker's virtual disk cannot grow	Free ≥ 15 GB (Trash, `~/Library/Caches`, Xcode `DerivedData`, `docker system prune`), then quit and reopen Docker Desktop
`bind: address already in use`	V1 stack or another app on the port	`docker compose down` in the V1 folder, or change the host port in `docker-compose.yml`
`migrate` exits 1	database not ready, or a migration file edited after it was applied	`docker compose logs migrate`; migrations are checksummed, so add a new file instead of editing
il-api unhealthy	PostgreSQL unreachable or wrong password	check `.env`, `docker compose logs il-api`
BigQuery requests return 503 BACKEND_UNAVAILABLE	emulator not running / GCP credentials invalid	`docker compose ps bigquery`; `curl -s localhost:8000/health/ready`
BigQuery pages feel slow	the emulator executes queries one at a time (SQLite)	expected locally (about 1–3 s per query); real BigQuery is similar per statement but runs statements in parallel
Callbacks not arriving	callback host not allow-listed	`CALLBACK_ALLOWED_HOSTS=wwmg-source,mycarehub`
401 from the IL	wrong key	keys are in `.env` (`WWMG_API_KEY`, `MYCAREHUB_API_KEY`, `ADMIN_API_KEY`)

11. Stop and reset

```
docker compose stop          # stop, keep data
docker compose down          # remove containers, keep volumes (pgdata, bqdata)
docker compose down -v       # remove containers AND data (fresh start next time)
```

Integration Layer V2.0: API Reference

Five HTTP services. Each one also serves interactive docs at `/docs` (Swagger UI) and the raw spec at `/openapi.json`; exported copies are in [`docs/openapi/`](openapi/).

Service	Base URL (local)	Used by
Integration Layer	`http://localhost:8000`	WWMG_Source, MyCareHub, console, il-scheduler
ConsumerApplication	`http://localhost:8001`	operators, console
WWMG_Source	`http://localhost:8002`	demo producer; receives callbacks
MyCareHub	`http://localhost:8003`	members (web UI); receives callbacks
Console backend	`http://localhost:8080/api`	the console web UI

1. Integration Layer

1.1 Authentication

Every `/v1/*` call sends `X-API-Key`. Keys map to a source application (`IL_API_KEYS`): `WWMG_Source:<key>,MyCareHub:<key>`. `/v1/admin/*` needs the admin key (`IL_ADMIN_API_KEY`). A source application can only read, list and cancel its own requests.

1.2 Submit a request

`POST /v1/requests` → 202 Accepted (200 when the idempotency key was seen before)

```
{
  "workflow_name": "care_request",
  "payload": {"member_id": "M-1001", "request_type": "appointment", "urgency": "routine", "preferred_date": \
    "2026-10-02"},
  "idempotency_key": "mch-7f3a",
  "callback_url": "http://mycarehub:8003/hooks/il",
  "correlation_id": "0b9f…",
  "priority": 0
}
```

```
{"request_id": "4c1e…", "correlation_id": "0b9f…", "status": "PENDING", "is_duplicate": false,
 "storage_backend": "bigquery", "status_url": "/v1/requests/4c1e…"}
```

The payload is validated against the workflow's `input_schema`; `storage_backend` shows where the request was stored (the workflow's current setting).

1.3 Read, list, cancel

Call	Response
`GET /v1/requests/{id}?include_input=true`	Full status: `status`, `success_flag`, `workflow_status`, `output_json`, `error_code`/`error_message`, retries, timestamps, callback state, `storage_backend`
`GET /v1/requests?status=&workflow_name=&storage_backend=&from_date=&to_date=&include_payload=&limit=`	The caller's requests from both backends, newest first
`POST /v1/requests/{id}/cancel?reason=`	Cancels a PENDING request (409 otherwise)

1.4 Workflows

Call	Purpose
`GET /v1/workflows`, `GET /v1/workflows/{name}`	Registered workflows with JSON Schemas and `storage_backend`
`PUT /v1/admin/workflows/{name}` (admin)	Create/update: `target_app`, `description`, `input_schema`, `output_schema`, `max_retries`, **`storage_backend`** (`postgres`/`bigquery`), `is_active`. A schema change bumps `schema_version`. 201 when created.
`POST /v1/admin/requests/{id}/replay` (admin)	Re-queue a FAILED request (either backend)

1.5 Errors

`{"error": {"code": "...", "message": "...", "errors": [{"path": "$.member_id", "message": "...", "rule": "pattern"}]}}`

HTTP	Code	When
401	UNAUTHORIZED	missing/invalid `X-API-Key`
404	NOT_FOUND	unknown id, or owned by another source app
409	NOT_CANCELLABLE / NOT_REPLAYABLE	wrong status
413	PAYLOAD_TOO_LARGE	body > 1 MB
422	REQUEST_INVALID	envelope invalid
422	UNKNOWN_WORKFLOW	not registered or inactive
422	INPUT_SCHEMA_INVALID	payload fails the JSON Schema (one entry per problem)
422	CALLBACK_HOST_NOT_ALLOWED	callback host not in `IL_CALLBACK_ALLOWED_HOSTS`
422	INVALID_JSON_SCHEMA	admin sent an invalid schema
503	BACKEND_UNAVAILABLE	the workflow's backend (BigQuery) cannot be reached

1.6 Callback contract

When a request reaches COMPLETED, FAILED or CANCELLED and has a `callback_url`, the IL sends:

```
POST <callback_url>
Content-Type: application/json
X-IL-Request-Id: 4c1e…
X-Correlation-Id: 0b9f…
X-IL-Attempt: 1
X-IL-Timestamp: 1790000000
X-IL-Signature: sha256=<hex HMAC-SHA256(secret, "<timestamp>.<raw body>")>

{"event": "request.completed", "request_id": "4c1e…", "correlation_id": "0b9f…", "workflow_name": \
    "care_request",
 "status": "COMPLETED", "success_flag": true, "workflow_status": "SCHEDULED", "storage_backend": "bigquery",
 "output_json": {"member_id": "M-1001", "decision": "SCHEDULED", "scheduled_for": "2026-10-02T10:30", \
    "message": "…", "reference": "CR-…"},
 "error": null, "received_at": "…", "completed_at": "…", "attempt": 1}
```

Answer any 2xx. Failures are retried after 10 s, 20 s, 40 s, … (max 30 min), 6 attempts. Verify the signature and reject timestamps older than 5 minutes (`ilcommon.il_client.verify_signature`).

1.7 Health

`GET /health` (liveness) · `GET /health/ready` → `{"status":"ready","database":"ok","bigquery":{"ok":true,"latency_ms":…,"target":"emulator:intlayer-local.integration"}}`

2. Workflow payloads

order_validation (WWMG_Source, PostgreSQL by default)

```
{"order_id": "ORD-1", "customer_id": "C-1", "currency": "EUR", "order_date": "2026-09-28",
 "items": [{"sku": "A", "quantity": 2, "unit_price": 10.0}]}
```

Output: `order_id`, `decision` (`APPROVED`/`REJECTED`), `total_amount`, `currency`, `reasons[]`.

care_request (MyCareHub, BigQuery by default)

```
{"member_id": "M-1001", "request_type": "appointment|prescription_refill|referral|lab_results",
 "urgency": "routine|soon|urgent", "preferred_date": "2026-10-02", "notes": "optional, ≤ 500 chars"}
```

Output: `member_id`, `request_type`, `decision` (`SCHEDULED`, `APPROVED`, `SUBMITTED`, `RELEASED`, `ESCALATED`), `message`, `reference`, and `scheduled_for` or `ready_by` when relevant. Notes containing `SIMULATE_OUTAGE` trigger a retryable failure (demo).

3. ConsumerApplication

Call	Purpose	
`GET /health`	`{"workers": {"postgres": true, "bigquery": true}}`	
`GET /health/ready`	PostgreSQL required; BigQuery reported	
`GET /stats`	Totals plus per-backend `running`, `paused`, `inflight`, counters (`claimed`, `completed`, `retried`, `failed`, `lease_lost`)	
`POST /worker/pause?backend=postgres|bigquery`	Stop claiming (all backends when `backend` is omitted); in-flight work finishes	
`POST /worker/resume?backend=…`	Resume	

4. WWMG_Source

Call	Purpose
`POST /orders?idempotency_key=&use_callback=true`	Submit an order (body = order payload) through the IL
`GET /orders/{request_id}`	Poll the IL
`POST /hooks/il`	Callback receiver (verifies `X-IL-Signature`)
`GET /hooks/il/received?request_id=`	Callbacks received (in memory)

5. MyCareHub

Call	Purpose
`GET /`	Member portal (web UI)
`GET /api/members`	Synthetic demo members
`POST /api/care-requests`	Body: `member_id`, `request_type`, `urgency`, `preferred_date?`, `notes?`. Returns `{"http_status": 202, "response": {...IL response...}}` or the IL error
`GET /api/care-requests?member_id=M-1001`	The member's requests with status, decision and message
`GET /api/care-requests/{id}` · `POST /api/care-requests/{id}/cancel`	Detail · cancel while pending

Call	Purpose
`POST /hooks/il`	Callback receiver → member notifications
`GET /api/notifications?member_id=`	Notifications created from callbacks

6. Console backend (`/api`)

Used by the console UI; runs server-side with the source apps' keys and the admin key.

Area	Calls			
Dashboard	`GET /api/overview` (both backends: KPIs, statuses, per-backend counts, hourly, services)			
Requests	`GET /api/requests?status=&workflow=&backend=&q=` · `GET /api/requests/{id}` (metadata, events, API calls, job) · `POST /api/requests` (send as `source_app`) · `POST /api/requests/{id}/cancel` · `…/replay`			
Workflows	`GET /api/workflows` · `PUT /api/workflows/{name}` (includes `storage_backend`)			
Payload library	`GET/POST /api/samples` · `PUT/DELETE /api/samples/{id}` · `POST /api/samples/validate` · `POST /api/templates/preview`			
Traffic jobs	`GET/POST /api/jobs` · `GET/PUT/DELETE /api/jobs/{id}` · `POST /api/jobs/{id}/run\`	pause\	resume · GET /api/job-runs/{run_id}`	
System jobs	`GET /api/system-jobs` · `POST /api/system-jobs/{name}/run\`	pause\	resume · PUT /api/system-jobs/{name}` (interval)	
ConsumerApplication	`POST /api/consumer/pause\`	resume?backend=`		
Analytics	`GET /api/analytics?range=1h\`	24h\	7d\	30d&workflow=&backend=&tz=&offset=`
Telemetry	`GET /api/telemetry?kind=all\`	state\	http\	job · GET /api/callbacks · GET /api/api-calls`

7. Endpoint index (generated)

Integration Layer (port 8000)

OpenAPI: `docs/openapi/integration-layer.json` · Swagger UI: `http://localhost:8000/docs`

Method	Path	Summary
GET	`/health`	Liveness
GET	`/health/ready`	Readiness — PostgreSQL must be reachable; BigQuery is reported (requests for BigQuery workflows fail with 503 without it).
POST	`/v1/requests`	Submit Request — Validates the payload against the workflow's input schema and stores it in the workflow's backend.
GET	`/v1/requests`	List Requests — The caller's requests from both backends, newest first.
GET	`/v1/requests/{request_id}`	Get Request — Status, result and callback state of one of the caller's requests (polling).
POST	`/v1/requests/{request_id}/cancel`	Cancel Request — Cancels a request that ConsumerApplication has not picked up yet.

Method	Path	Summary
GET	`/v1/workflows`	List Workflows
GET	`/v1/workflows/{workflow_name}`	Get Workflow
POST	`/v1/admin/requests/{request_id}/replay`	Replay Request — Re-queues a FAILED request (any source app).
PUT	`/v1/admin/workflows/{workflow_name}`	Upsert Workflow

ConsumerApplication (port 8001)

OpenAPI: `docs/openapi/consumer-application.json` · Swagger UI: `http://localhost:8001/docs`

Method	Path	Summary
GET	`/health`	Liveness
GET	`/health/ready`	Readiness — PostgreSQL must be reachable; BigQuery is reported.
GET	`/stats`	Stats — Totals across backends, plus per-backend detail.
POST	`/worker/pause`	Pause — Stop claiming new requests (all backends, or ?backend=postgres/bigquery). In-flight work finishes.
POST	`/worker/resume`	Resume

WWMG_Source (port 8002)

OpenAPI: `docs/openapi/wwmg-source.json` · Swagger UI: `http://localhost:8002/docs`

Method	Path	Summary
POST	`/orders`	Submit Order — Forward an order to the Integration Layer (workflow: order_validation).
GET	`/orders/{request_id}`	Order Status — Polling mode: ask the Integration Layer for the current status.
POST	`/hooks/il`	Receive Callback — Callback mode: the Integration Layer POSTs the final result here.
GET	`/hooks/il/received`	Received Callbacks
GET	`/health`	Health

MyCareHub (port 8003)

OpenAPI: `docs/openapi/mycarehub.json` · Swagger UI: `http://localhost:8003/docs`

Method	Path	Summary
GET	`/api/members`	Members
POST	`/api/care-requests`	Create Care Request — Submits a care request through the Integration Layer. Returns its HTTP status and body.
GET	`/api/care-requests`	List Care Requests
GET	`/api/care-requests/{request_id}`	Care Request
POST	`/api/care-requests/{request_id}/cancel`	Cancel
POST	`/hooks/il`	Receive Callback — Signed callback from the Integration Layer when a care request finishes.
GET	`/api/notifications`	Notifications
GET	`/health`	Health

Console backend (port 8080)

OpenAPI: `docs/openapi/console.json` · Swagger UI: `http://localhost:8080/docs`

Method	Path	Summary
GET	`/api/overview`	Overview
GET	`/api/requests`	List Requests
POST	`/api/requests`	Send Request
GET	`/api/requests/{request_id}`	Request Detail
POST	`/api/requests/{request_id}/cancel`	Cancel
POST	`/api/requests/{request_id}/replay`	Replay
GET	`/api/workflows`	Workflows
PUT	`/api/workflows/{name}`	Upsert Workflow
GET	`/api/samples`	Samples
POST	`/api/samples`	Create Sample
PUT	`/api/samples/{sample_id}`	Update Sample
DELETE	`/api/samples/{sample_id}`	Delete Sample
POST	`/api/samples/validate`	Validate Sample
POST	`/api/templates/preview`	Preview
GET	`/api/jobs`	Jobs
POST	`/api/jobs`	Create Job
GET	`/api/jobs/{job_id}`	Job Detail
PUT	`/api/jobs/{job_id}`	Update Job
DELETE	`/api/jobs/{job_id}`	Delete Job
GET	`/api/job-runs/{run_id}`	Job Run
POST	`/api/jobs/{job_id}/run`	Run Job
POST	`/api/jobs/{job_id}/pause`	Pause Job
POST	`/api/jobs/{job_id}/resume`	Resume Job
GET	`/api/system-jobs`	System Jobs
POST	`/api/system-jobs/{name}/run`	Run System Job
POST	`/api/system-jobs/{name}/pause`	Pause System Job
POST	`/api/system-jobs/{name}/resume`	Resume System Job
PUT	`/api/system-jobs/{name}`	Reschedule System Job
POST	`/api/consumer/{action}`	Consumer Control — Pause/resume ConsumerApplication's workers (all, or one backend's queue).
GET	`/api/analytics`	Analytics
GET	`/api/telemetry`	Telemetry
GET	`/api/callbacks`	Callbacks — Callbacks received by WWMG_Source and MyCareHub, newest first.
GET	`/api/api-calls`	Api Calls
GET	`/health`	Health